

Metody Programownia: Reference

Data: 2026-06-18

Zakres: 1 - 2

1. Setup środowiska developerskiego

Do programowania w OCaml najpierw potrzebujemy ustawić nasze środowisko developerskie. Poniżej najważniejsze kroki do wykonania.

Instalacja OPAM, wg systemu operacyjnego.

Zaktualizowanie OPAM

```
opam update
```

Zainicjalizowanie menedżera zasobów OPAM

```
opam init --bare -a -y
```

Dodanie instalacji (switcha) OCamla z odpowiednimi zależnościami i wersją kompilatora

```
opam switch create cs3110-2026sp ocaml-base-compiler.5.3.0
```

Teraz wyloguj się z systemu operacyjnego, lub zrestartuj komputer.

Po wpisaniu `opam switch list` powinieneś otrzymać dokładnie takie wyjście:

```
~ % opam switch list
# switch      compiler
description
cs3110-2026sp ocaml-base-compiler.5.3.0,ocaml-options-vanilla.1 ocaml-
base-compiler = 5.3.0
```

Uwaga: Jeśli zobaczysz ostrzeżenie

```
[WARNING] The environment is not in sync with the current switch.
You should run: eval $(opam env)
```

, musisz ręcznie skonfigurować swój interfejs powłoki (ang. text interface shell).

Zainstalowanie zależności potrzebnych do wykładu

```
opam install -y utop odoc ounit2 qcheck bisect_ppx menhir ocaml-lsp-server
ocamlformat
```

To nie są wszystkie zależności. Niektóre, jak `csv` mogą pojawić się na pracowni. Instalujemy je wtedy podobnie, np.

```
opam install -y csv
```

2. Typy danych w OCamlu

Podstawowe typy danych

Liczby całkowite

```
_ : int = 3
```

Liczby zmiennoprzecinkowe

```
_ : float = 3.14
```

Wartości algebry Boole’a

```
_ : bool = true
```

Znaki wypisywane

```
_ : char = 'a'
```

Ciągi znaków wypisywalnych

```
_ : string = "Hello World"
```

Wbudowane struktury danych

Krotki

Deklaracja typu: iloczyn kartezjański typów na odpowiednich pozycjach

Konstruktor: poszczególne składowe oddzielone przecinkiem

Destruktor: tak jak konstruktor

Przykłady:

```
_ : int * int = (4,2)
_ : int * string = 4,"dwa"
_ : int * string * float = 4,"dwa",3.14
_ : (int * int) * int = (1,2),3

(* destruktory *)

match (4,2) with (fst, snd) -> ...
```

Tablice

(nie są używane na przedmiocie)

Deklaracja typu: typ elementów po czym następuje `array`

Konstruktor: elementy oddzielone średnikiem (;), obłożone pałąką (!), obłożone nawiasami kwadratowymi ([])

Destruktor: pozyskiwanie konkretnego, i-tego elementu przez `.(i)` na końcu

Przykłady:

```
_ : int array = [| 4; 2; 1 |]
_ : int = [| 4; 2; 1 |].(2) (* -> 1 *)
```

Listy

Deklaracja typu: typ elementów po czym następuje `list`

Konstruktory:

- wykorzystanie operatora `cons` typu `'a -> 'a list -> 'a list`, który dokłada element z lewej strony na początek listy. Złożoność: $O(1)$.
- elementy oddzielone średnikami (;), obłożone nawiasami kwadratowymi ([])

Destruktor: wykorzystywanie operatora `cons` czyli `::`.

Przykłady:

```
_ : int list = [ 4; 2; 1 ]
_ : int list = 4 :: 2 :: 1 :: []
_ : int = match [ 4; 2; 1 ] with four :: rest -> four
```

Funkcje

Funkcja jest podstawowym typem wartości w OCamlu. Typ określamy jako `'a -> 'b`. Funkcje można składać, wykorzystując popularny lukier syntaktyczny przez wymianę argumentów po spacji, np. `fun a b -> a + b`, to tak naprawdę `fun a -> fun b -> a + b`.

Deklaracja typu: typ argumentu $\rightarrow$ typ wyjścia

Konstruktor: lambda, składnia: `fun <argument> -> <wyrażenie>`

Destruktor: wywołanie funkcji na argumentach typu `'a`.

Przykłady:

```
(* typ funkcji: int -> int *)
(fun a -> a * a) 2
|_ (2 * 2)
|_ 4

(* typ funkcji: int -> int -> int *)
(fun a b -> a + b) 2 5
|_ (fun a -> fun b -> a + b) 2 5
|_ (fun b -> 2 + b) 5
|_ (2 + 5)
|_ 7
```

Funkcje można definiować za pomocą skróconej składni `let`:

```
let add a b = a + b zamiast let add = fun a b -> a + b.
```

3. Wiązanie zmiennych

Wiązanie odbywa za pomocą słowa kluczowego `let`.

np. `let a = 5`

Aby zawęzić zakres danej zmiennej, możemy użyć słowa kluczowego `in`, w ten sposób:

```
let a = 5 in a + 10 (* wyewaluuje się do 15 *)

let b = c + 10 (* BŁĄD! Niezwiązana zmienna 'c'! *)
```

Możemy zagnieżdżać wiązania:

```
let a = 2 in
  let b = 40 in
    let c = a + b in
      c

(* całe to wyrażenie wyewaluuje się do 42 *)
```

oraz wiązać wiele zmiennych jednocześnie

```
let a = 2 and b = 40 in
  a + b

let a = 2 and b = a + 38 in (* BŁĄD! Niezwiązana zmienna 'a'! *)
  a + b

(* Wyjaśnienie: wyrażenie a + 38 nadal jest w tym samym zakresie (scope) co
samo let, więc żadne 'a' nie zostało jeszcze związane. *)
```

Jak wspomnieliśmy wyżej, istnieje lukier syntaktyczny na definiowanie funkcji.

Zamiast `let add = fun a b -> a + b` możemy zapisać `let add a b = a + b`. Zauważmy, że związanie `let add a = fun b -> a + b` również odnosi się do tej samej funkcji.

4. Funkcje i warunki

Funkcje rekurencyjne

Funkcje mogą być rekurencyjne. Aby skorzystać z identyfikatora funkcji, należy zastąpić `let` parą słów `let rec`:

```
let rec fib x =
  fib (x - 1) + fib (x - 2)
```

Aby funkcja ta uległa kiedyś terminacji, musimy dodać przypadek bazowy funkcji. Użyjemy do tego instrukcji warunkowej `if`. Działa ona jak w każdym innym języku programowania.

```
let rec fib x =
  if x = 0 || x = 1 then
    1
  else
    fib (x - 1) + fib (x - 2)
```

Funkcje mogą być wzajemne rekurencyjne. Wtedy należy dwa związania rekurencyjne `let rec` połączyć w jedno słowem kluczowym `and`:

```
let rec fun_a a =
  if a = 0 then 42
  else fun_b (a/2)

and
fun_b b =
  if b = 0 then 13
  else fun_a (b/2)
```

Funkcje z akumulatorem

Akumulatorem nazywamy argument który jest stanem funkcji przekazywanym w głąb rekurencyjnych wywołań funkcji. Na początku jest inicjalizowany elementem neutralnym danej operacji, np. 1 dla mnożenia, 0 dla dodawania.

```
let rec factorial x acc =
  if x = 0 then acc
  else factorial (x-1) (x*acc)

let fac_10 = factorial 10 1
```

Funkcje mogą być rekurencyjne **ogonowo**, tzn. wywołują inną funkcję (lub siebie samą) wyłącznie ze zmodyfikowanymi argumentami, bez wykorzystywania wartości otrzymanej przez wywoływaną funkcję. Funkcje rekurencyjne ogonowo są o wiele bardziej wydajne od tych nie ogonowych, ponieważ OCaml jest w stanie zwolnić stos wywołań funkcji podmieniając obecnie wykonywaną funkcję na tą, do której ciała wskazuje.

```
(* Funkcja nie ogonowa *)
let rec factorial x =
  if x = 0 then 1
  (* tutaj wynik factorial jest jeszcze mnożony przez x *)
  else x * factorial (x-1)
```

Funkcją rekurencyjnie ogonową jest funkcja z akumulatorem przykład wyżej.

Pattern Matching

Pozwala na dopasowywanie wzorców do danego wyrażenia i warunkową ewaluację wyrażeń. Kolejne wzorce wypisujemy po znaku pałki (|). Wzorce te są porównywane do wyrażenia od góry do dołu.

```
let is_zero x =
  match x with
  | 0 -> true
  | _ -> false
```

`_` służy jako wildcard, dopasowuje każdy przypadek który został.

Czasami kolejność ułożenia wzorców może mieć znaczenie, jeśli któryś jest słabszy niż inny. Szczególnie funkcja poniżej zawsze będzie zwracać `false`, ponieważ do `_` matchuje się każde wyrażenie.

```
let is_zero x =
  match x with
  | _ -> false
  | 0 -> true
```

Matching można rozumieć jako taki `switch` z `C`, lub `match` z Pythona, albo też jako bardzo rozbudowany `if`.

Matching poza kontrolą warunkową programu pozwala nam dekonstruować struktury danych.

```
let hd xs =
  match xs with
  | x :: xs -> x
```

OCaml poinformuje nas, że `match` w powyższym kodzie nie jest wyczerpujący. Tak faktycznie jest, ponieważ typem `::` jest `'a -> 'a list -> 'a list`, więc wymaga on `'a`. Jeśli lista jest pusta (`[]`), to OCaml nie znajdzie żadnego elementu który mógłby scons-ować, by otrzymać listę pustą, zatem musimy ten przypadek również rozpatrzyć:

```
let hd xs =
  match xs with
  | x :: xs -> x
  | [] -> failwith "wywołano hd na pustej liście"
```

`failwith` wyrzuca wyjątek, korzystamy z niego szczególnie jeśli jakiś przypadek jest nieosiągalny, lub nie jesteśmy w stanie

spełnić oczekiwań użytkownika, tak jak u góry biorąc głowę z pustej listy. Wkrótce poznamy typ `'a option` i drugi przypadek będzie obsługowany w inny sposób.

Nic nie stoi na przeszkodzie, aby matchować na bardziej złożonych wyrażeniach. Np. jeśli dany element jest równy jakiemuś wyrażeniu.

Poniższa funkcja sprawdza, czy przekazana lista składa się wyłącznie z powtarzającej sekwencji `1;2;3` .

```
let rec is_123 xs =  
  match xs with  
  | 1 :: 2 :: 3 :: rest -> is_123 rest  
  | [] -> true  
  | _ -> false
```